Math Quest 3D

Complete Unity C# Implementation for Class 1 to Class 12 Curriculum

1. System Overview & Architecture

Module	Component	Description
Core Engine	MathQuestManager.cs	Controls turn flow, score tracking, grade-level difficulty scaling, and game loop.
Mechanics	MathGenerator.cs	Generates age-appropriate mathematical problems dynamically across Grades 1 through 12.
User Interface	UIBattleController.cs	Manages dynamic health bars, input parsing, dynamic feedback, and Developer Info panel.
UI Entities	HealthBarUI.cs	Smoothly animates slider values for player and monster health state updates.

2. Dynamic Math Question Generator

```
using UnityEngine;

public enum ProblemType { Addition, Subtraction, Multiplication, Division,
Algebra, Trigonometry, Calculus }

public struct MathQuestion
```

```
{
    public string questionText;
    public float correctAnswer;
    public float[] options;
}
public static class MathGenerator
{
    public static MathQuestion GenerateQuestion(int gradeLevel)
    {
        MathQuestion q = new MathQuestion();
        q.options = new float[4];
        if (gradeLevel <= 3) // Basic Arithmetic (Class 1 - 3)
        {
            int a = Random.Range(1, 20);
            int b = Random.Range(1, 20);
            bool isAdd = Random.value > 0.5f;
            if (isAdd)
            {
                q.questionText = $"{a} + {b} = ?";
                q.correctAnswer = a + b;
```

```
        }
        else
        {
            q.questionText = $"{a + b} - {b} = ?";
            q.correctAnswer = a;
        }
    }
    else if (gradeLevel <= 6) // Multiplication & Division (Class 4 - 6)
    {
        int a = Random.Range(2, 12);
        int b = Random.Range(2, 12);
        bool isMult = Random.value > 0.5f;
        if (isMult)
        {
            q.questionText = $"{a} × {b} = ?";
            q.correctAnswer = a * b;
        }
        else
        {
            q.questionText = $"{a * b} ÷ {b} = ?";
```

```
        q.correctAnswer = a;
    }
}
else if (gradeLevel <= 9) // Algebra & Power Equations (Class 7 - 9)
{
    int x = Random.Range(1, 10);
    int m = Random.Range(2, 5);
    int c = Random.Range(1, 15);
    int rhs = m * x + c;
    q.questionText = $"Solve for x: {m}x + {c} = {rhs}";
    q.correctAnswer = x;
}
else // Advanced Math, Trigonometry & Calculus (Class 10 - 12)
{
    int mode = Random.Range(0, 3);
    if (mode == 0) // Trigonometry
    {
        int[] angles = { 0, 30, 45, 60, 90 };
        int selectedAngle = angles[Random.Range(0, angles.Length)];
        q.questionText = $"Evaluate: sin({selectedAngle}°)";
```

```
                q.correctAnswer = Mathf.Round(Mathf.Sin(selectedAngle *
Mathf.Deg2Rad) * 100f) / 100f;

            }

            else if (mode == 1) // Differentiation

            {

                int power = Random.Range(2, 5);

                int coeff = Random.Range(1, 6);

                int evalAt = Random.Range(1, 4);

                // d/dx (c * x^n) = c * n * x^(n-1)

                q.questionText = $"d/dx [{coeff}x^{power}] at x = {evalAt}";

                q.correctAnswer = coeff * power * Mathf.Pow(evalAt, power -
1);

            }

            else // Basic Integration

            {

                int power = Random.Range(1, 4);

                // Integral of x^n from 0 to 2

                q.questionText = $"∫ (x^{power}) dx from 0 to 2";

                q.correctAnswer = Mathf.Round((Mathf.Pow(2, power + 1) /
(power + 1)) * 100f) / 100f;

            }

        }
```

```
        // Generate Multiple Choice Distractors
        q.options[0] = q.correctAnswer;
        for (int i = 1; i < 4; i++)
        {
            float offset = Random.Range(1, 5) * (Random.value > 0.5f ? 1 :
-1);
            q.options[i] = q.correctAnswer + offset;
        }
        // Shuffle Options
        for (int i = 0; i < q.options.Length; i++)
        {
            float temp = q.options[i];
            int randomIndex = Random.Range(i, q.options.Length);
            q.options[i] = q.options[randomIndex];
            q.options[randomIndex] = temp;
        }
        return q;
    }
}
```

3. Smooth Health Bar Display Component

```
using UnityEngine;

using UnityEngine.UI;

public class HealthBarUI : MonoBehaviour

{

    [SerializeField] private Slider healthSlider;

    [SerializeField] private Image fillImage;

    [SerializeField] private Color fullHealthColor = Color.green;

    [SerializeField] private Color lowHealthColor = Color.red;

    private float targetHealth = 1f;

    private float lerpSpeed = 5f;

    private void Update()

    {

        if (healthSlider != null)

        {

            healthSlider.value = Mathf.Lerp(healthSlider.value, targetHealth,
Time.deltaTime * lerpSpeed);

            if (fillImage != null)

            {

                fillImage.color = Color.Lerp(lowHealthColor, fullHealthColor,
healthSlider.value);
```

```
            }
        }
    }
    public void SetHealth(float currentHealth, float maxHealth)
    {
        targetHealth = Mathf.Clamp01(currentHealth / maxHealth);
    }
}
```

4. Main Math Quest Battle Manager

```
using System.Collections;
using UnityEngine;
using UnityEngine.UI;
public class MathQuestManager : MonoBehaviour
{
    [Header("Game Parameters")]
    public int currentGradeLevel = 1; // Default Class 1
    public float maxPlayerHealth = 100f;
    public float maxMonsterHealth = 100f;
    private float currentPlayerHealth;
    private float currentMonsterHealth;
```

```
private MathQuestion currentQuestion;

[Header("UI Component References")]

public Text questionText;

public Text feedbackText;

public Text gradeDisplayText;

public Text developerInfoText;

public Button[] answerButtons;

public Dropdown gradeDropdown;

public HealthBarUI playerHealthBar;

public HealthBarUI monsterHealthBar;

public GameObject devPanel;

private void Start()

{

    InitializeDeveloperInfo();

    InitializeGradeDropdown();

    ResetBattle();

}

private void InitializeDeveloperInfo()

{

    if (developerInfoText != null)
```

```
    {
        developerInfoText.text = "<b>Math Quest 3D</b>\n" +
                                 "Developed by: <b>Dipanshu Rajak</b>\n" +
                                 "Target Audience: Class 1 to 12\n" +
                                 "Engine: Unity 3D (C#)";
    }
}
private void InitializeGradeDropdown()
{
    if (gradeDropdown != null)
    {
        gradeDropdown.clearOptions();
        var options = new System.Collections.Generic.List<string>();
        for (int i = 1; i <= 12; i++)
        {
            options.Add("Class " + i);
        }
        gradeDropdown.AddOptions(options);
        gradeDropdown.onValueChanged.AddListener(OnGradeChanged);
    }
}
```

```
    public void OnGradeChanged(int index)

    {

        currentGradeLevel = index + 1;

        if (gradeDisplayText != null)

        {

            gradeDisplayText.text = "Selected Level: Class " +
currentGradeLevel;

        }

        ResetBattle();

    }

    public void ResetBattle()

    {

        currentPlayerHealth = maxPlayerHealth;

        currentMonsterHealth = maxMonsterHealth;

        if (playerHealthBar) playerHealthBar.SetHealth(currentPlayerHealth,
maxPlayerHealth);

        if (monsterHealthBar) monsterHealthBar.SetHealth(currentMonsterHealth,
maxMonsterHealth);

        if (feedbackText != null) feedbackText.text = "A wild Math Monster
appears! Answer correctly to strike!";

        NextTurn();

    }
```

```
    private void NextTurn()

    {

        if (currentMonsterHealth <= 0)

        {

            if (feedbackText) feedbackText.text = "<b>VICTORY!</b> You
defeated the Math Monster!";

            SetButtonsInteractable(false);

            return;

        }

        if (currentPlayerHealth <= 0)

        {

            if (feedbackText) feedbackText.text = "<b>DEFEAT!</b> The monster
overcame your defenses!";

            SetButtonsInteractable(false);

            return;

        }

        currentQuestion = MathGenerator.GenerateQuestion(currentGradeLevel);

        if (questionText != null) questionText.text =
currentQuestion.questionText;

        for (int i = 0; i < answerButtons.Length; i++)

        {
```

```
            if (i < currentQuestion.options.Length)

            {

                answerButtons[i].gameObject.SetActive(true);

                Text btnText =
answerButtons[i].GetComponentInChildren<Text>();

                if (btnText) btnText.text =
currentQuestion.options[i].ToString("0.##");

                int index = i;

                answerButtons[i].onClick.RemoveAllListeners();

                answerButtons[i].onClick.AddListener(() =>
SubmitAnswer(currentQuestion.options[index]));

            }

            else

            {

                answerButtons[i].gameObject.SetActive(false);

            }

        }

        SetButtonsInteractable(true);

    }

    public void SubmitAnswer(float selectedValue)

    {

        SetButtonsInteractable(false);
```

```
        bool isCorrect = Mathf.Approximately(selectedValue,
currentQuestion.correctAnswer);

        if (isCorrect)

        {

            float damage = 25f;

            currentMonsterHealth -= damage;

            if (monsterHealthBar)
monsterHealthBar.SetHealth(currentMonsterHealth, maxMonsterHealth);

            if (feedbackText) feedbackText.text =
"<color=green>CORRECT!</color> You dealt " + damage + " damage!";

        }

        else

        {

            float damage = 15f;

            currentPlayerHealth -= damage;

            if (playerHealthBar)
playerHealthBar.SetHealth(currentPlayerHealth, maxPlayerHealth);

            if (feedbackText) feedbackText.text =
"<color=red>INCORRECT!</color> Correct answer was: " +
currentQuestion.correctAnswer.ToString("0.##") + ". Monster hit you for " +
damage + " damage!";

        }

        StartCoroutine(TurnDelayRoutine());
```

```
}

private IEnumerator TurnDelayRoutine()

{

    yield return new WaitForSeconds(2.0f);

    NextTurn();

}

private void SetButtonsInteractable(bool state)

{

    foreach (Button btn in answerButtons)

    {

        if (btn != null) btn.interactable = state;

    }

}

public void ToggleDeveloperPanel()

{

    if (devPanel != null)

    {

        devPanel.SetActive(!devPanel.activeSelf);

    }

}
```

}